

Prof. Dr. Matthias O. Franz

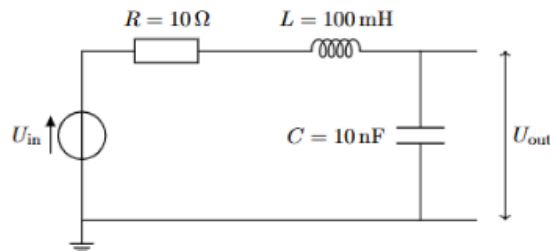
LINEARE ZEITINVARIANTE SYSTEME

Versuch 4, Praktikum Technische Grundlagen der künstlichen Intelligenz

In diesem Versuch geht es um lineare zeitinvariante Systeme: in Teil 1 führen wir eine Systemidentifikation anhand eines simulierten Schwingkreises durch, in Teil 2 benutzen wir FFT-Filter zum Aufbau einer digitalen Frequenzweiche.

1. RLC-Reihenschwingkreis

Ein Reihenschwingkreis besteht aus den drei Elementen: ohmscher Widerstand R , Kapazität C und Induktivität L , die in Reihe angeordnet sind.



Das Eingangssignal ist eine zeitveränderliche Spannung U_{in} (in unserem Fall erzeugt durch den Funktionsgenerator). Das Ausgangssignal ist die Spannung, die am Kondensator abfällt, und wird entsprechend an den beiden Kondensatoranschlüssen hinausgeführt bzw. gemessen. Typische Anwendungsfälle solcher Schwingkreise sind Oszillatoren oder Filter im Radio- und Funkbereich. Der RLC-Reihenschwingkreis ist ein zeitinvariantes lineares System. Ihre erste Aufgabe ist es, den Frequenzgang dieses Systems zu ermitteln. Wenden Sie dazu die Methode aus Vorlesung 11, Folie 18 bis 22 an.

1. Bauen Sie den Schaltkreis mit den zur Verfügung gestellten Bauteilen auf. Das Oszilloskop wird mit den beiden Anschlüssen des Kondensators verbunden. Am Funktionsgenerator wählen Sie eine Rechteckschwingung mit Frequenz 100 Hz und stellen diese so ein, dass der untere Spannungspegel bei 0 V liegt, und der obere bei 1 V (warum?). Einstellungen am Funktionsgenerator: FREQ = 100.0 Hz, AMPL = 500.0 mVpp und DC Offset = 0.250 V_{DC}.

2. Stellen Sie Ihr Oszilloskop so ein, dass auf dem ersten Kanal das Eingangssignal und auf dem zweiten Kanal das Ausgangssignal dargestellt wird, jeweils nur eine Periode. Speichern Sie die gemessenen Signale als CSV-Datei ab.
3. Lesen Sie die CSV-Datei in ein Jupyter-Notebook ein und schneiden Sie aus diesen Daten möglichst genau die Sprungantwort heraus. In dieser Form ist die Sprungantwort noch stark verrauscht, daher muss diese geglättet werden, bevor man sie numerisch differenzieren kann. Benutzen Sie dazu einen gleitenden Mittelwertfilter mit Fensterbreite 50 und die Numpy-Funktion *convolve*.
4. Erzeugen Sie daraus die Impulsantwort des Schwingkreises durch numerische Differentiation (z.B. mit *numpy.gradient*).
5. Durch Anwendung der Fouriertransformation auf die so berechnete Impulsantwort erhalten Sie den Frequenzgang des Systems. Stellen Sie den Amplituden- und Phasengang als Bode-Diagramme dar (Frequenzachse in HZ). Wo liegt die Resonanzfrequenz des Schwingkreises? Vergleichen Sie Ihr Ergebnis mit dem theoretisch vorhergesagten Wert

$$f_{\text{res}} = \frac{1}{2\pi\sqrt{L \cdot C}}.$$

2. Digitale Frequenzweiche mit FFT-Filtern

Eine Frequenzweiche teilt ein Audiosignal auf mehrere Lautsprecher in Abhängigkeit von der Tonhöhe (Frequenz) auf. Hochwertige Lautsprecher wie Hochtonlautsprecher und Tieftonlautsprecher sind konstruktiv bedingt nur für einen engen Frequenzbereich optimiert. Die Kombination mehrerer unterschiedlicher Lautsprechertypen in Lautsprecherboxen für verschiedene Frequenzbereiche benötigt entsprechende Frequenzweichen.

In Teil 2 implementieren wir eine digitale Frequenzweiche mit FFT-Filtern. Als Beispiel können Sie einen zehnstündigen Ausschnitt aus einem Musikstück Ihrer Wahl verwenden, das natürlich ein möglichst breites Tonspektrum aufweisen sollte. Wir verwenden zum Einlesen und Analysieren der Soundfiles das Paket *librosa*, das Sie dafür auf Ihrem System installieren müssen. Beispiele zur Anwendung von *librosa* finden Sie im zweiten Teil des Notebooks zur Vorlesung 9.

Vorgehensweise:

- a. Lesen Sie Ihr Musikstück mit *librosa* ein und ermitteln Sie die Abtastrate. Wählen Sie einen geeigneten 10s-Ausschnitt und stellen Sie diesen als Kurve und als Spektrogramm dar.
- b. Zerlegen Sie Ihr Signal in Blöcke der Länge 1024.
- c. Tieftöner haben typischerweise einen Frequenzbereich von 30 Hz – 600 Hz, Mitteltöner 300 Hz – 5.000 Hz und Hochtöner 2.000 Hz – 20.000 Hz. Wählen Sie eine geeignete Grenzfrequenz für einen Tiefpass zur Ansteuerung des Tieftöners und für einen Hochpass zur Ansteuerung des Hochtöners und rechnen Sie diese in die zugehörige

Wellenzahl um. Der Bandpass zur Ansteuerung des Mitteltöners liegt folglich zwischen diesen beiden Grenzfrequenzen.

- d. Implementieren drei FFT-Filter: (1) einen idealen Tiefpass; (2) einen idealen Bandpass; (3) einen idealen Hochpass. Dies geschieht durch einfaches Nullsetzen aller Frequenzen im Sperrbereich und Beibehaltung des Spektrums im Durchlassbereich für jeden 1024er-Block. Achten Sie darauf, dass das FFT-Spektrum spiegelsymmetrisch ist, d.h. z.B. bei einem Hochpass müssen Sie das Spektrum für alle Frequenzen von 0 bis zur Grenzwellenzahl k auf Null setzen, aber auch für alle Frequenzen von $N - k$ bis $N - 1$.
- e. Überprüfen Sie Ergebnis akustisch durch Abspielen mit `IPython.display.Audio`.

WICHTIGE ANMERKUNG

1. Solltest du nicht alle diese Methoden an einem Ort haben wollen, kannst du sie rum schieben wie du lustig bist
2. Es kann sein das manche Methoden die hier eingefügt wurden nicht 1 zu 1 den ansprüchen aus der Aufgabe erfüllen. Die Methoden sind aber leicht anpassbar.
3. Sollte unklar sein was die Methoden machen, gerne mich Fragen oder ChatGPT
4. Solltest du es verbessern können, gerne verbessern
5. i love u

In [24]: *# Viele der Methoden wurden bereits in den vorherigen Versuchen verwendet, daher*

```
import sys
import pandas as pd
import matplotlib.pyplot as plt
import pyaudio
import numpy as np
import os
import re
from scipy.signal import find_peaks, chirp, spectrogram
from IPython.display import Audio
import librosa
from IPython.display import Audio

# conv für CVS
def conv(x):
    return float(x.replace(',', '.'))

# Plot functions
def plotstart():
    plt.figure(figsize=(10,4))
```

```

# Finalisiert den Plot: entfernt doppelte Legendeneinträge, setzt Titel und Achs
# aktiviert Gitternetz, platziert die Legende oben rechts, optimiert das Layout

def plotend(ueberschrift, xlabel, ylabel, grid=True):
    handles, labels = plt.gca().get_legend_handles_labels()
    by_label = dict(zip(labels, handles))
    plt.legend(by_label.values(), by_label.keys())
    plt.title(f" {ueberschrift}")
    plt.xlabel(f"{xlabel}")
    plt.ylabel(f"{ylabel}")
    if(grid == True): plt.grid(True, which='both')
    plt.minorticks_on()
    plt.legend(loc='upper right') #Legende oben rechts
    plt.tight_layout()
    plt.show()

# Methode zur CSV einlesung
def plot(x, y):
    plt.plot(x, y) # weniger Punkte

# Berechnet und plottet die Fouriertransformation
def fft(u, t):
    # Fouriertransformation mit numpy.fft.fft()
    U = np.fft.fft(u)

    # Amplitude
    A = np.abs(U)

    M = len(u) # anz abtastwerte
    dt = (t[1] - t[0])/1000 # Abtastintervall

    # Frequenzachse
    n = np.arange(M)
    f = n / (M * dt)

    # erste Hälfte
    amplitudenspektrum = A[:M//2] # Amplitudenspektrum
    frequenzen = f[:M//2] # Frequenzen

    plot(frequenzen, amplitudenspektrum)
    plotend(ueberschrift="Amplitudenspektrum (FFT)", xlabel="Frequenz in Herz",

# Darstellung vom Amplituden-Spektrum eines Signalwertes
def ampspec(signalwert, zeitachse):
    # Abtastintervall
    dt = zeitachse[1] - zeitachse[0]
    fs = 1 / dt

    # FFT
    U = np.fft.fft(signalwert)
    A = np.abs(U)

    M = len(signalwert)

    # Frequenzachse richtig
    freqs = np.fft.fftfreq(M, d=dt)

    # nur positive Frequenzen

```

```

mask = freqs >= 0

plotstart()
plot(freqs[mask], A[mask])
plotend(ueberschrift="Amplitudenspektrum (FFT)",
        xlabel="Frequenz in Hz",
        ylabel="Amplitude")

# Gauss Windowing Methode
def gauss_window_fft(signal):
    fs = 44100 # Frequency sample
    L = 512 # Windowlänge

    # 1. Gaußfenster erzeugen
    sigma = L / 4 # Fensterbreite
    mu = (L - 1) / 2 # Mittelpunkt
    gauss_window = np.exp(-0.5 * ((np.arange(L) - mu) / sigma)**2)

    # 2. Fensterung (50 % Overlap)
    step = L // 2
    segments = []

    for start in range(0, len(signal) - L + 1, step):
        segment = signal[start:start + L]
        segment = segment * gauss_window # Gauß-Fenster anwenden
        segments.append(segment)

    segments = np.array(segments)

    # 3. FFT jedes Fensters --> Leistungsspektrum
    spectra = np.abs(np.fft.fft(segments, axis=1))**2

    # 4. Leistungsspektren mitteln (über alle Fenster)
    mean_power = spectra.mean(axis=0)

    # 5. Wurzel -> Amplitudenspektrum
    amplitude_spectrum = np.sqrt(mean_power)

    # 6. Frequenzachse erzeugen
    freqs = np.fft.fftfreq(L, d=1 / fs)

    # nur positive Frequenzen
    pos = freqs >= 0
    f_pos = freqs[pos]
    A_pos = amplitude_spectrum[pos]

    return f_pos, A_pos

```

Aufgabe 1

3. Lesen Sie die CSV-Datei in ein Jupyter-Notebook ein und schneiden Sie aus diesen Daten möglichst genau die Sprungantwort heraus. In dieser Form ist die Sprungantwort noch stark verrauscht, daher muss diese geglättet werden, bevor man sie numerisch differenzieren kann. Benutzen Sie dazu einen gleitenden Mittelwertfilter mit Fensterbreite 50 und die Numpy-Funktion *convolve*.

```
In [25]: def cut_step(t, u, t_start, t_end):
        """
        Schneidet ein Zeitintervall aus und setzt die Zeit neu bei 0.
        """
        mask = (t >= t_start) & (t <= t_end)
        t_cut = t[mask] - t[mask][0]
        u_cut = u[mask]
        return t_cut, u_cut

In [26]: def readAndPlotCSV(dateiname, SprungAnt = 1, plott=0, plottzusatz = ""):
        data = np.genfromtxt(
            dateiname,
            delimiter=';',
            skip_header=2,
            converters={0: conv, 1: conv, 2: conv},
            autostrip=True
        )

        # Spalten aufteilen
        t = data[:, 0] / 1000.0      # Zeit: ms → s
        u_in = data[:, 1]           # Kanal A (Eingang)
        u_out = data[:, 2]          # Kanal B (Ausgang = Sprungantwort)

        # Zeit bei 0 starten lassen
        t = t - t[0]

        if(SprungAnt == 1):
            t_start = 0.010        # 10 ms → 2. Sprung
            t_end = 0.015          # 3 ms danach
            t_cut, u_cut = cut_step(t, u_out, t_start, t_end)

            t = t_cut
            u_out = u_cut

        if plott == 1:
            # Plot: Ausgangssignal (Sprungantwort)
            plotstart()
            plt.plot(t, u_out)
            if(SprungAnt == 1):
                plotend("Sprungantwort RLC (Kanal B)", "Zeit [s]", "Spannung [V]")
            else:
                plotend("Original RLC (Kanal B)", "Zeit [s]", "Spannung [V]")

        return t, u_out
```

```
In [27]: def signalPlott(t, u_out, SprungAnt = 1):

        plotstart()
        plt.plot(t, u_out)
        if(SprungAnt == 1):
```

```

        plotend("Sprungantwort RLC (Kanal B) geglättet", "Zeit [s]", "Spannung [V]")
    else:
        plotend("Original RLC (Kanal B) geglättet", "Zeit [s]", "Spannung [V]")

```

```

In [28]: def smooth(time, signal, window = 50, plott = 0, save = 0, SprungAnt = 1):
        kernel = np.ones(window) / window
        # Array aus 1en so lang wie window/window
        # => summe aller Werte = 1
        # gleiche Gewichtung
        smoothSignal = np.convolve(signal, kernel, mode='same')
        # Führt eine Faltung (Convolution) zwischen dem Signal
        # und dem Kernel aus

        # Für jede Position wird der Mittelwert der
        # benachbarten windows berechnet

        # mode='same':
        # Ausgabe hat die gleiche Länge wie das Eingangssignal
        if (plott == 1):
            signalPlott(time, smoothSignal, SprungAnt)

        if (save == 1):
            np.savetxt(
                "Data/signal.csv",
                np.column_stack((time, smoothSignal)),
                delimiter=';',
                header="time; signal",
                comments=''
            )

        return smoothSignal

```

```

In [29]: def readAndSmooth(dateiname = "Data/Reihenschwingkreis.csv", plott = 0, save = 0,
        time, u_out = readAndPlotCSV(dateiname, plott = plott, SprungAnt = SprungAnt)

        u_smooth = smooth(time, u_out, window=50, plott=plott, save=save, SprungAnt
        return time, u_out, u_smooth

```

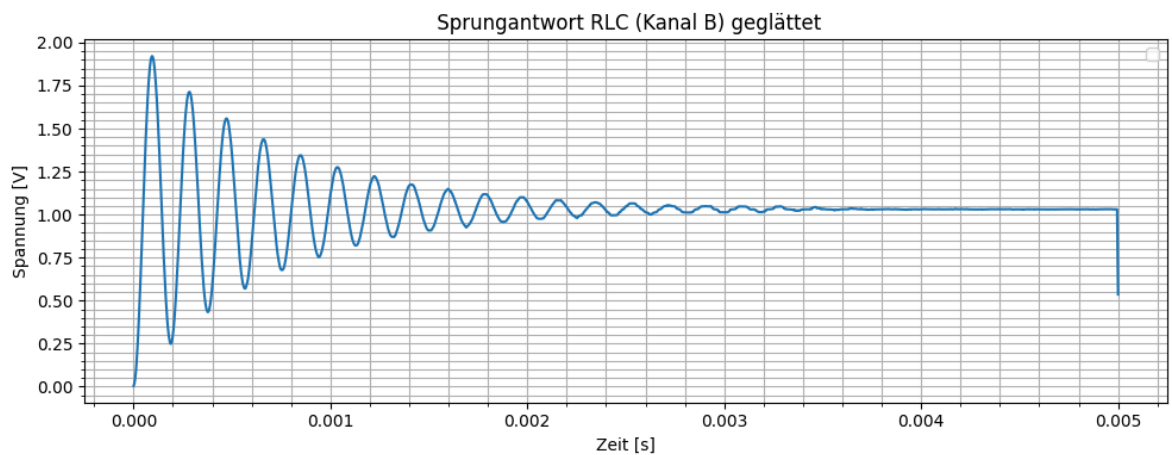
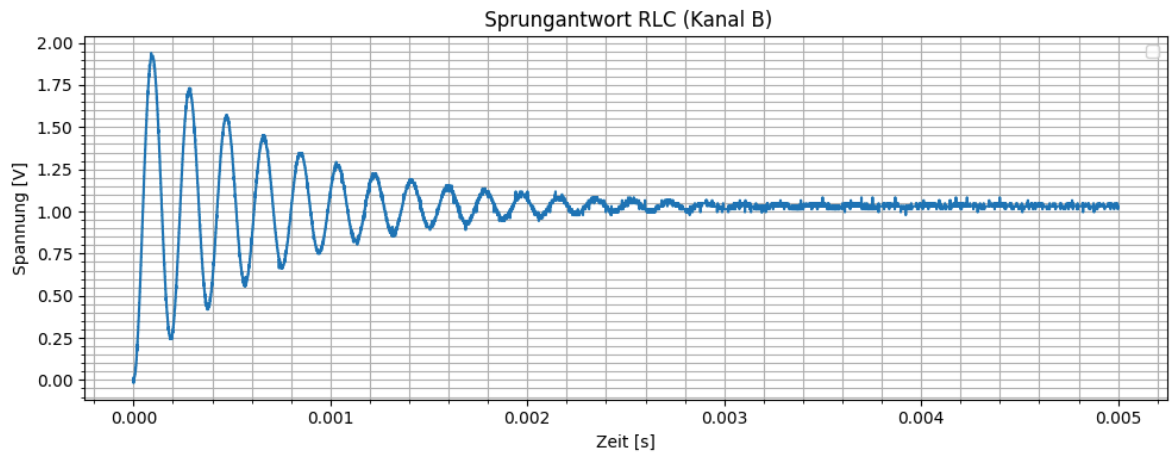
```

In [30]: # Nur Sprungausschnitt
        print("Nur Sprungausschnitt")
        time, u_out, u_smooth = readAndSmooth(plott=1, SprungAnt = 1)
        # Kompletter Ausschnitt
        print("Kompletter Ausschnitt")
        time, u_out, u_smooth = readAndSmooth(plott=1, SprungAnt = 0)

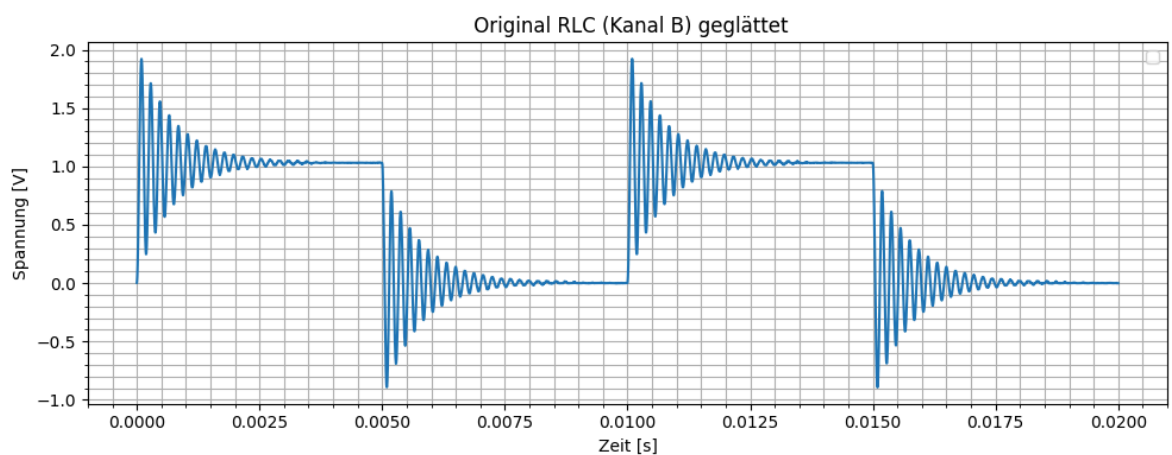
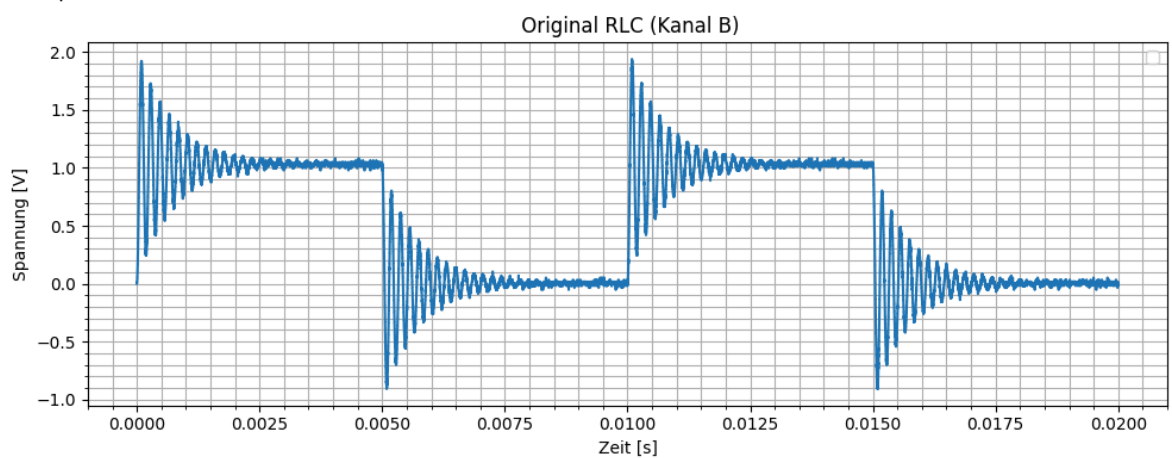
```

Nur Sprungausschnitt

C:\Users\Nikita\AppData\Local\Temp\ipykernel_5920\2100571738.py:36: UserWarning:
No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend(loc='upper right') #legende oben rechts



Kompletter Ausschnitt



4. Erzeugen Sie daraus die Impulsantwort des Schwingkreises durch numerische Differentiation (z.B. mit *numpy.gradient*).

$$h(t) \approx \frac{u(t + \Delta t) - u(t - \Delta t)}{2\Delta t}$$

```
In [31]: def impulsantwort():
time, u_out, u_smooth = readAndSmooth(plott=1)
imAnt = np.gradient(u_smooth, time)

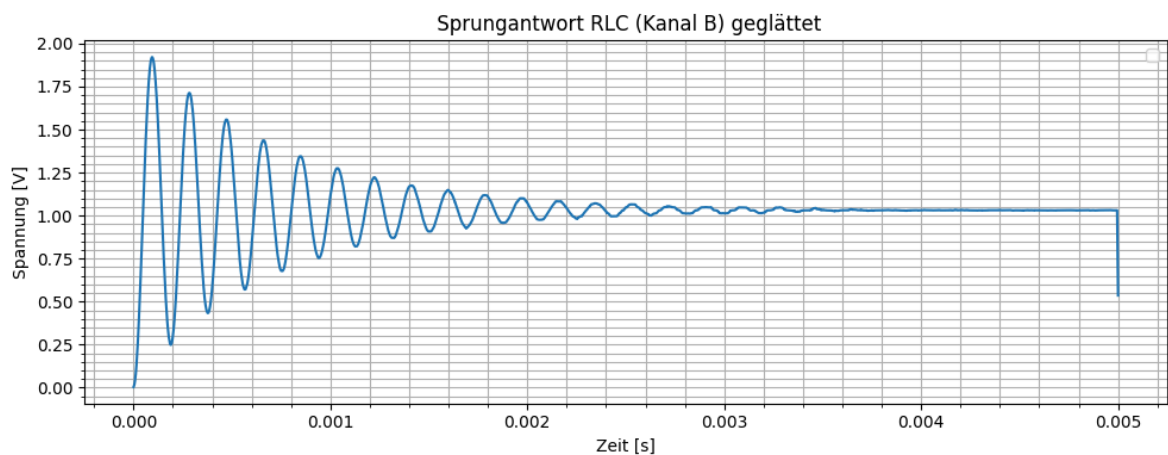
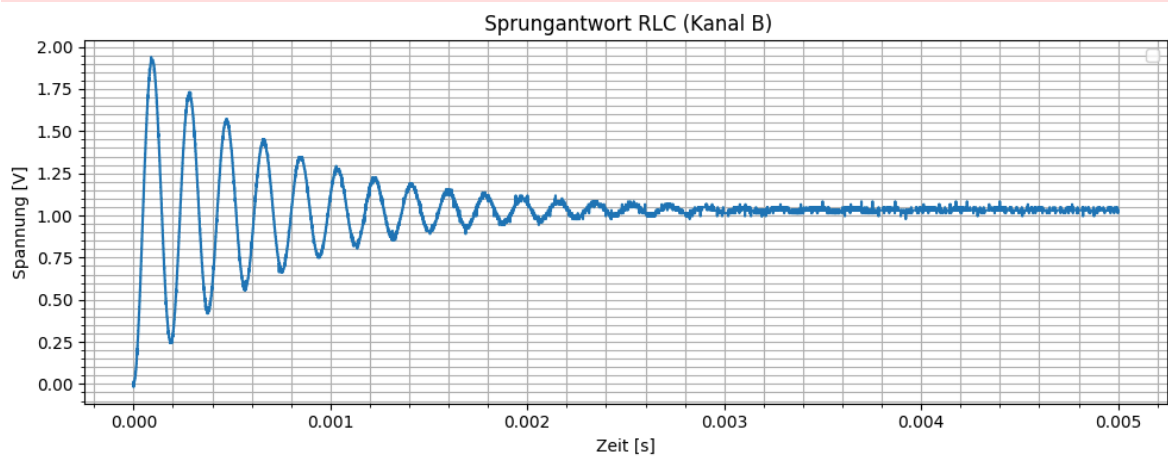
# Randartefakt entfernen
time = time[:-25]
imAnt = imAnt[:-25]

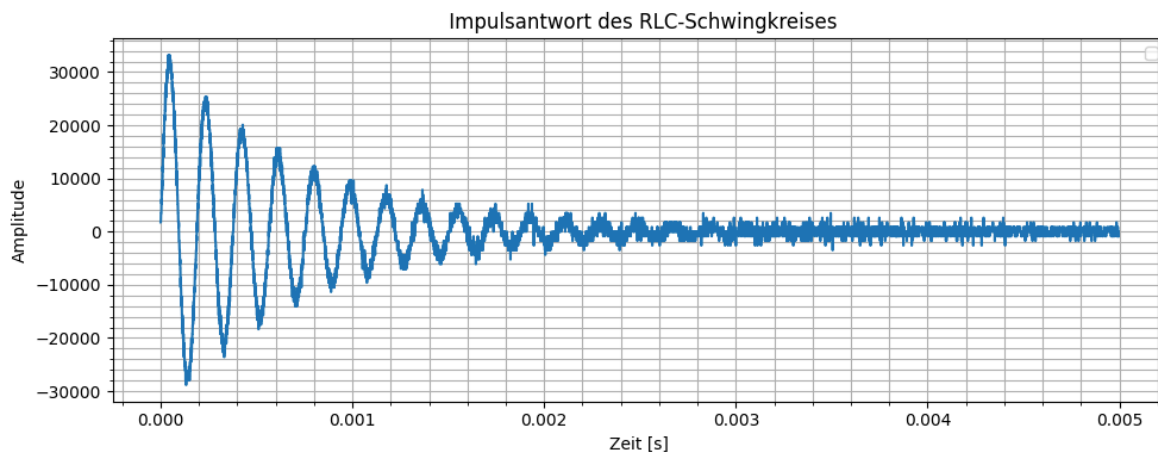
plotstart()
plot(time, imAnt)
plotend("Impulsantwort des RLC-Schwingkreises",
        "Zeit [s]",
        "Amplitude")

return time, imAnt
```

```
In [32]: impulsantwort()
```

C:\Users\Nikita\AppData\Local\Temp\ipykernel_5920\2100571738.py:36: UserWarning:
No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend(loc='upper right') #legende oben rechts



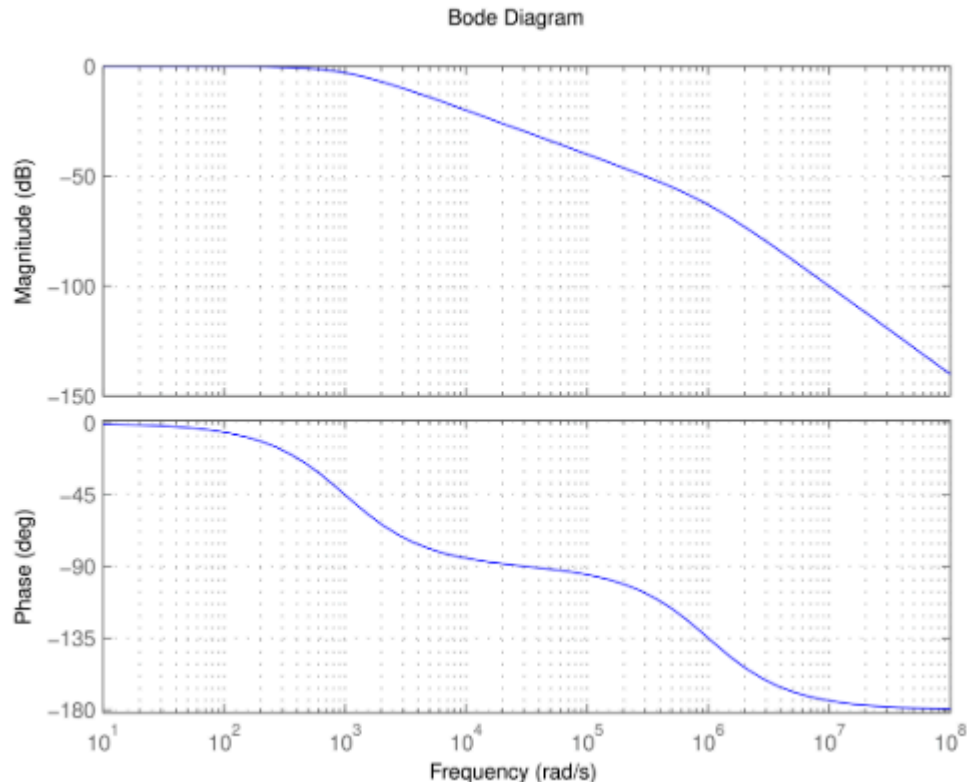


```
Out[32]: (array([0.0000000e+00, 2.0000000e-07, 4.0000000e-07, ..., 4.9943997e-03,
        4.9945997e-03, 4.9947997e-03], shape=(24975,)),
array([ 1.74704700e+03, 1.74704700e+03, 1.74704700e+03, ...,
        4.65661287e-10, 0.00000000e+00, -8.73524000e+02], shape=(24975,)))
```

5. Durch Anwendung der Fouriertransformation auf die so berechnete Impulsantwort erhalten Sie den Frequenzgang des Systems. Stellen Sie den Amplituden- und Phasengang als Bode-Diagramme dar (Frequenzachse in HZ). Wo liegt die Resonanzfrequenz des Schwingkreises? Vergleichen Sie Ihr Ergebnis mit dem theoretisch vorhergesagten Wert

$$f_{\text{res}} = \frac{1}{2\pi\sqrt{L \cdot C}}.$$

Beispiel wie so ein Bode-Diagramm aussieht:



```
In [33]: # Erstellt ein Bode-diagramm
# frequenz in Hz
# Amplitude in DB
# Phase in rad/s
def bode_diagramm(frequenz, amplitude, phase):
```

```

"""
frequenz      : Frequenzachse in Hz
amplitude_db  : Amplitudengang in dB
phase_rad     : Phasengang in Radiant
"""

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

# Amplitudengang
ax1.semilogx(frequenz, amplitude)
ax1.set_ylabel("Amplitude [dB]")
ax1.grid(True)

# Phasengang
ax2.plot(frequenz, phase)
ax2.set_ylabel("Phase [rad]")
ax2.set_xlabel("Frequenz [Hz]")
ax2.grid(True)

plt.tight_layout()
plt.show()

```

```

In [34]: '''
TODO: Fouriertransformation auf die Impulsantwort Anwenden für den Frequenzgang,
TODO: Ermittelte Resonanzfrequenz mit der theoretischen vorhersage vergleichen (
       wobei: L = Induktivität der Spule (in Henry, H), C Kapazität des Kondensat
...
def frequenzgang_und_resonanz(plot_bode=True):

    R = 10          # Ohm
    L = 100e-3      # 100 mH = 0.1 H
    C = 10e-9       # 10 nF

    # Impulsantwort holen
    time, imAnt = impulsantwort()

    # FFT der Impulsantwort
    dt = time[1] - time[0]
    H = np.fft.fft(imAnt)
    freqs = np.fft.fftfreq(len(imAnt), d=dt)

    # nur positive Frequenzen
    mask = freqs > 0
    f = freqs[mask]
    H = H[mask]

    # Amplitude (dB) und Phase
    amplitude_db = 20 * np.log10(np.abs(H))
    phase = np.angle(H) # rad

    # Resonanzfrequenz (experimentell)
    idx = np.argmax(np.abs(H))
    f_res_exp = f[idx]

    # Resonanzfrequenz (theoretisch)
    f_res_theo = 1 / (2 * np.pi * np.sqrt(L * C))
    print(f_res_theo)
    print(f_res_exp)
    # Optional: Bode-Diagramm
    if plot_bode:

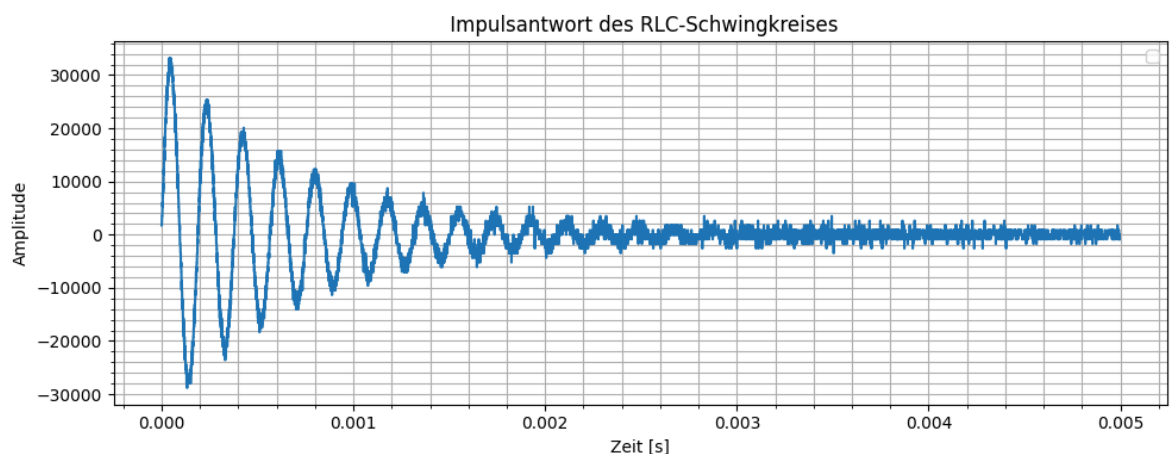
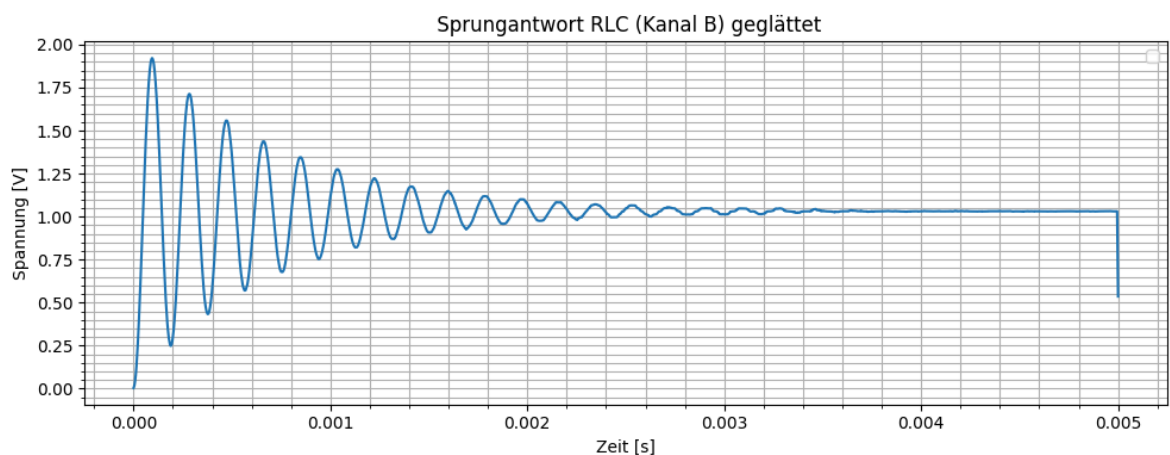
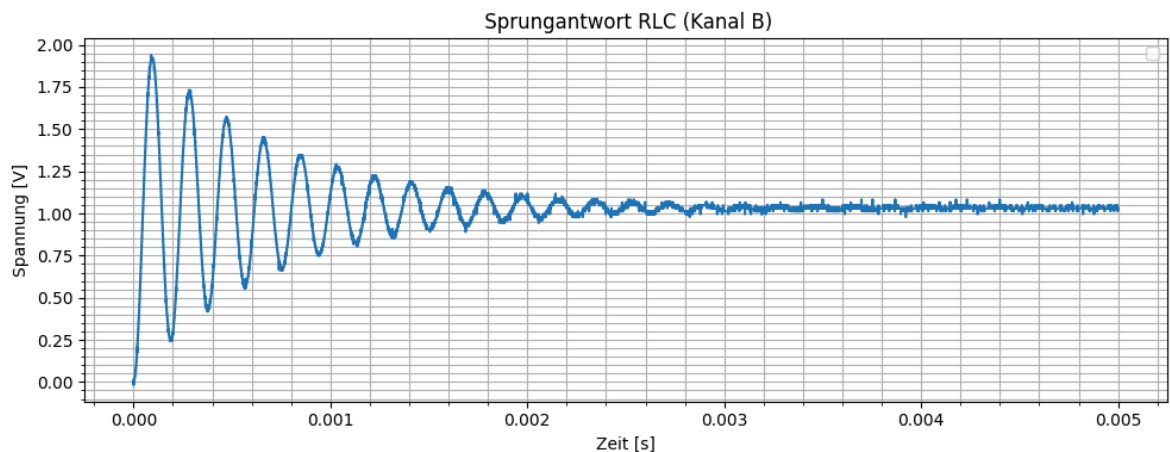
```

```
bode_diagramm(f, amplitude_db, phase)

return f, amplitude_db, phase, f_res_exp, f_res_theo
```

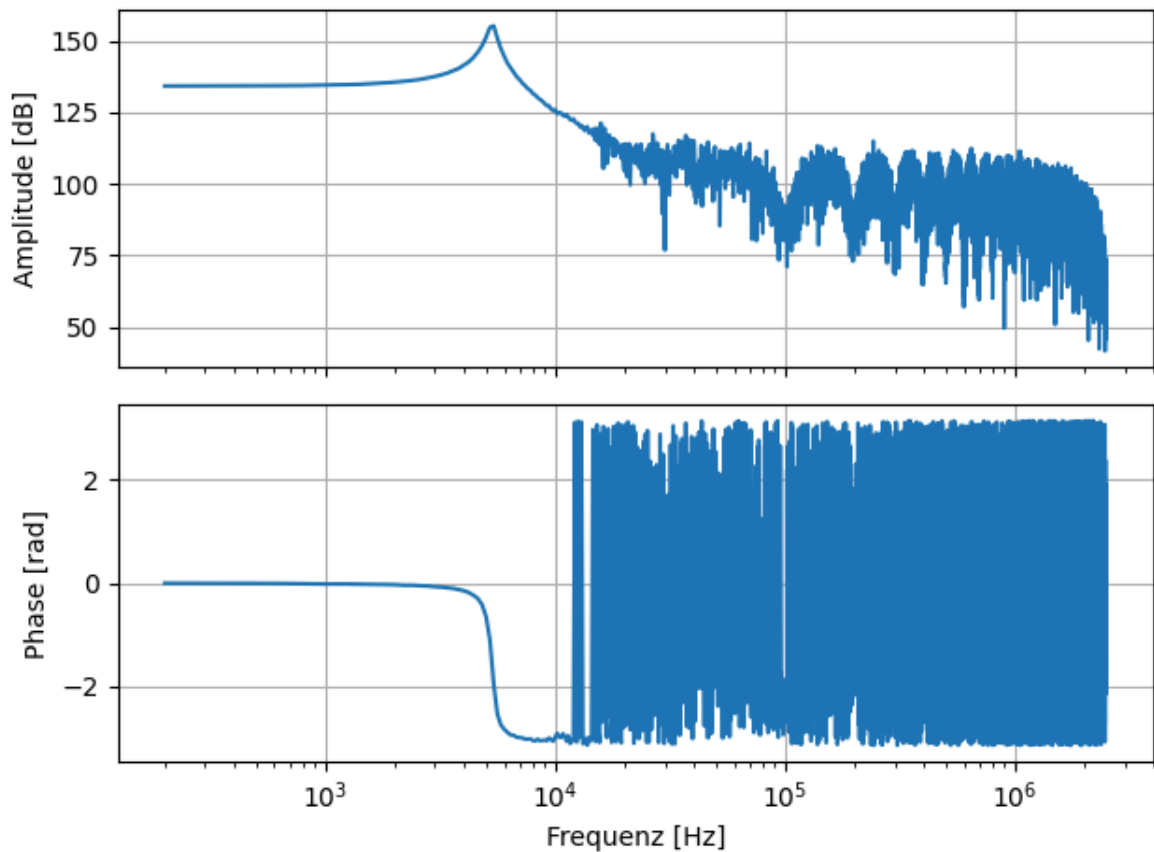
```
In [35]: frequenzgang_und_resonanz()
```

C:\Users\Nikita\AppData\Local\Temp\ipykernel_5920\2100571738.py:36: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
 plt.legend(loc='upper right') #legende oben rechts



5032.921210448703

5405.405405437507



```
Out[35]: (array([2.00200200e+02, 4.00400400e+02, 6.00600601e+02, ...,
                2.49949950e+06, 2.49969970e+06, 2.49989990e+06], shape=(12487,)),
          array([134.24490504, 134.28207552, 134.35919792, ..., 64.84348354,
                64.84622092, 64.84488714], shape=(12487,)),
          array([-3.05177037e-03, -5.59665438e-03, -7.02528342e-03, ...,
                -3.64079004e-04, -1.17713674e-05, -2.00621762e-04], shape=(12487,)),
          np.float64(5405.405405437507),
          np.float64(5032.921210448703))
```

Aufgabe 2

- a. Lesen Sie Ihr Musikstück mit librosa ein und ermitteln Sie die Abtastrate. Wählen Sie einen geeigneten 10s-Ausschnitt und stellen Sie diesen als Kurve und als Spektrogramm dar.

```
In [36]: def cut(audio, fs, plot=False):
...
    audio: Das Audio signal das man durch librosa bekommt
    ! Der Abschnitt darf nur 10 sekunden lang sein !
...
    start_time = 1*60 + 55 # 115 s
    end_time   = 2*60 + 5   # 125 s

    start_sample = int(start_time * fs)
    end_sample   = int(end_time * fs)

    audio_cut = audio[start_sample:end_sample]
```

```

print(len(audio_cut)/fs)

if plot:
    # Um das Signal sichtbarer zu machen wird nur jedes 100ste sample geplottet
    step = 100          # nur jedes 100. Sample
    t = np.arange(len(audio_cut)) / fs

    plt.figure(figsize=(20,5))
    plt.plot(t[::step], audio_cut[::step])
    plt.xlabel("Zeit [s]")
    plt.ylabel("Amplitude")
    plt.title("Audioausschnitt 1:55-2:05 (downsampled)")
    plt.grid(True)
    plt.show()

    return audio_cut

# Plottet ein gauss-spectrogramm. Für den fall dass man ein anderes Spektrogramm
def gauss_spectrogram(signal):
    fs = 44100
    L = 512

    # Gaußfenster wie oben
    sigma = L / 4
    mu = (L - 1) / 2
    gauss_window = np.exp(-0.5 * ((np.arange(L) - mu) / sigma)**2)

    # 50% Overlap
    noverlap = L // 2
    nfft = L

    # Spektrogramm berechnen
    f, t, Sxx = spectrogram(signal, fs, window=gauss_window,
                             nperseg=L, noverlap=noverlap, nfft=nfft)

    # Plotten
    plotstart()
    plt.pcolormesh(t, f, 10 * np.log10(Sxx))
    plt.colorbar(label="Spektraldichte [dB/Hz]")
    plotend("Gauß-Spektrogramm", "Zeit [S]", "Frequenz [Hz]", grid=False)

def std_spectrogram(signal, fs):
    plotstart()
    plt.specgram(signal, NFFT=1024, Fs=fs, noverlap=512)
    plt.xlabel("Zeit [s]")
    plt.ylabel("Frequenz [Hz]")
    plt.title("Spektrogramm (Hann-Fenster)")
    plt.colorbar(label="Spektraldichte [dB/Hz]")
    plt.show()

def playAudioFilename(filename, plot=True):
    audio, fs = librosa.load(filename, sr=None, mono=True)
    T = len(audio) / fs

    print("Audio shape:", audio.shape)
    print("Abtastfrequenz:", fs, "Hz")
    print("Dauer:", T, "s")

```

```

if plot:
    t = np.arange(len(audio)) / fs
    plt.figure(figsize=(20, 5))
    plt.plot(t, audio)
    plt.xlabel("Zeit [s]")
    plt.ylabel("Amplitude")
    plt.title("Audiosignal (Zeitbereich)")
    plt.grid(True)
    plt.show()

    return Audio(data=audio, rate=fs), audio, fs

def playAudio(audio, fs):
    return Audio(data = audio, rate=fs)

```

```

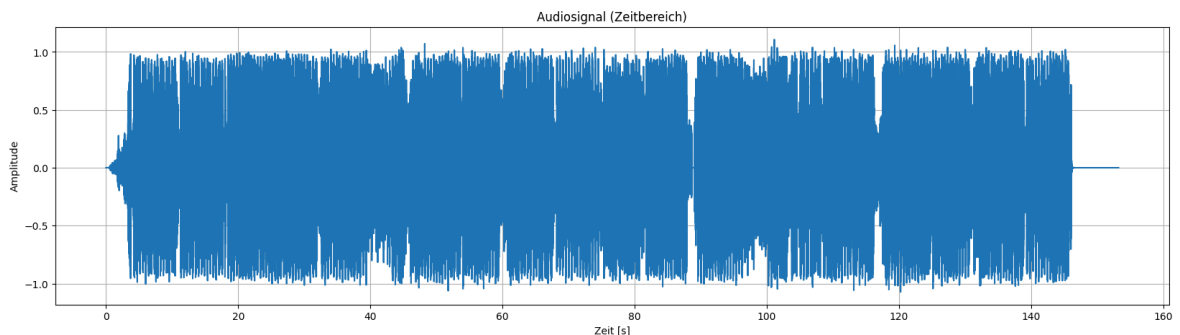
In [37]: '''
TODO: Schreibe eine Routine welche das Musikstück einliest, welches im DATA Ordner
TODO: Schneide ein 10-Sekunden abschnitt raus
TODO: Schreibe eine Plot Routine, welche dieses 10-Sekunden Stück als Kurve und
'''

file = "Data/Kenshi Yonezu - IRIS OUT.mp3"

print("So sieht das Audio Signal aus:")
player, audio, fs = playAudioFilename(file, plot=True)
gauss_spectrogram(audio)
std_spectrogram(audio, fs)
display(player)

```

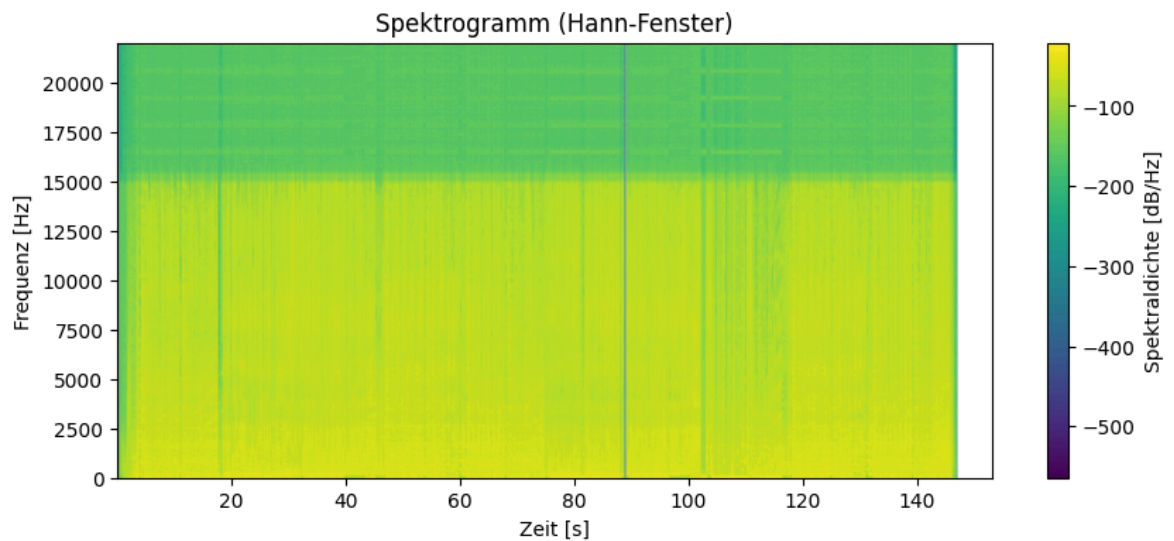
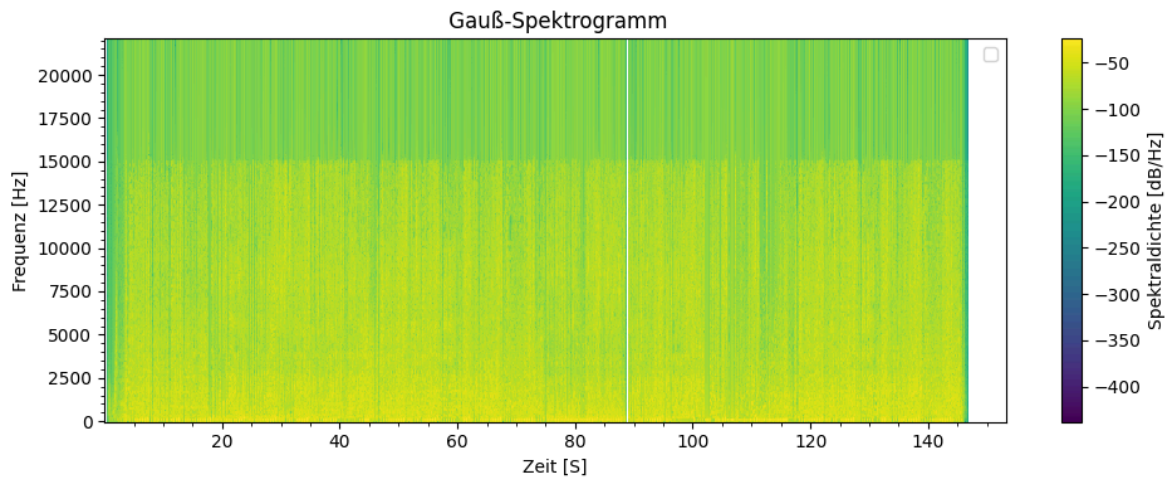
So sieht das Audio Signal aus:
 Audio shape: (6756352,)
 Abtastfrequenz: 44100 Hz
 Dauer: 153.20526077097506 s



```

C:\Users\Nikita\AppData\Local\Temp\ipykernel_5920\4293027023.py:52: RuntimeWarning:
divide by zero encountered in log10
    plt.pcolormesh(t, f, 10 * np.log10(Sxx))
C:\Users\Nikita\AppData\Local\Temp\ipykernel_5920\2100571738.py:36: UserWarning:
No artists with labels found to put in legend. Note that artists whose label starts
with an underscore are ignored when legend() is called with no argument.
    plt.legend(loc='upper right') #legende oben rechts

```

▶ 0:00 / 2:33 — 🔊 ⋮

```
In [38]: # Step 1, cut zhe Audio in a 10 seconds clip
cut_audio = cut(audio=audio, fs=fs)
cut_player = playAudio(cut_audio, fs)
display(cut_player)

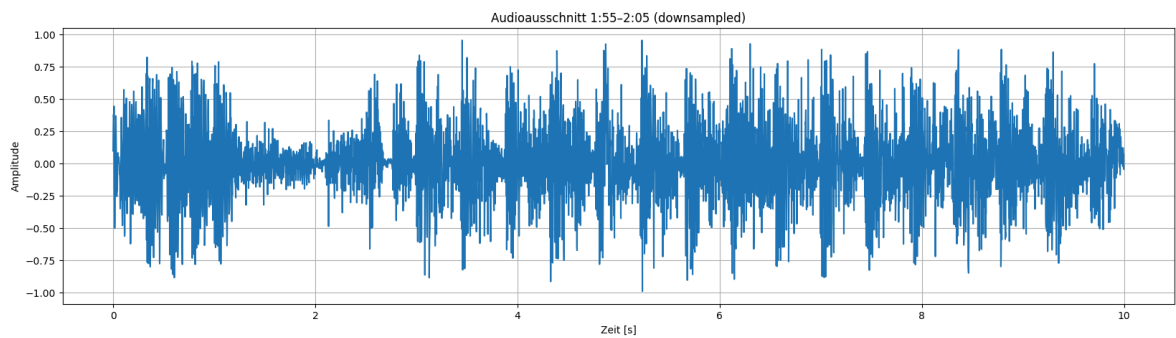
# Step 2, plot it als Kurve and alz zhe Spektrogramm
cut_audio = cut(audio=audio, fs=fs, plot=True)
gauss_spectrogram(cut_audio)
std_spectrogram(cut_audio, fs)

# Done
```

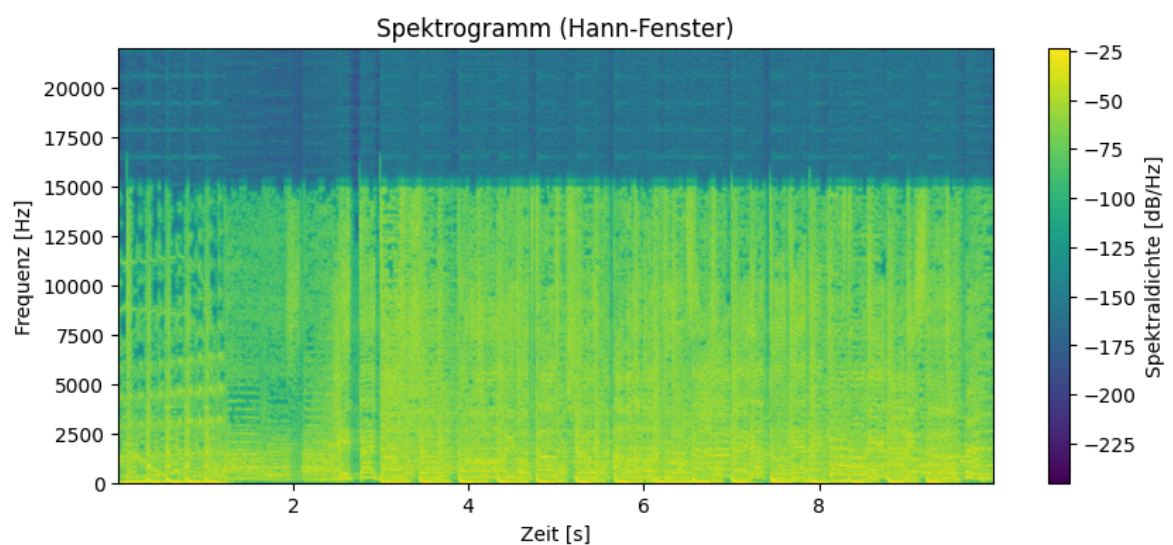
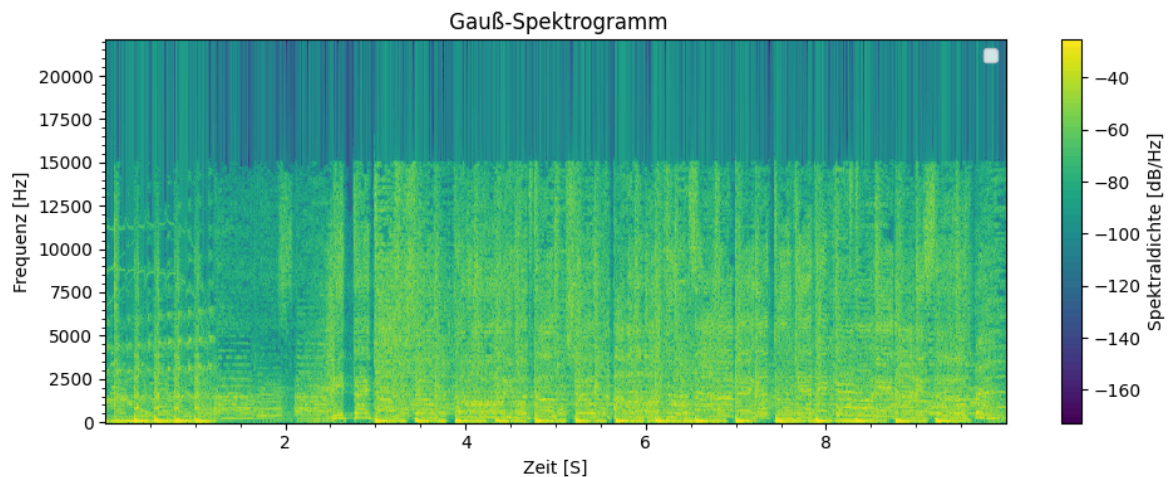
10.0

▶ 0:00 / 0:10 — 🔊 ⋮

10.0



C:\Users\Nikita\AppData\Local\Temp\ipykernel_5920\2100571738.py:36: UserWarning:
No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend(loc='upper right') #legende oben rechts



b. Zerlegen Sie Ihr Signal in Blöcke der Länge 1024.

```
In [39]: block_size = 1024
num_blocks = len(cut_audio) // block_size

blocks = cut_audio[:num_blocks * block_size]
blocks = blocks.reshape(num_blocks, block_size) # 1D Array, wir zu 2D. 430 Blöcke

blocks.shape
# (Anzahl_Blöcke, Länge)
```

Out[39]: (430, 1024)

- c. Tieftöner haben typischerweise einen Frequenzbereich von 30 Hz – 600 Hz, Mitteltöner 300 Hz – 5.000 Hz und Hochtöner 2.000 Hz – 20.000 Hz. Wählen Sie eine geeignete Grenzfrequenz für einen Tiefpass zur Ansteuerung des Tieftöners und für einen Hochpass zur Ansteuerung des Hochtöners und rechnen Sie diese in die zugehörige

Wellenzahl um. Der Bandpass zur Ansteuerung des Mitteltöners liegt folglich zwischen diesen beiden Grenzfrequenzen.

- Blocklänge $N = 1024$

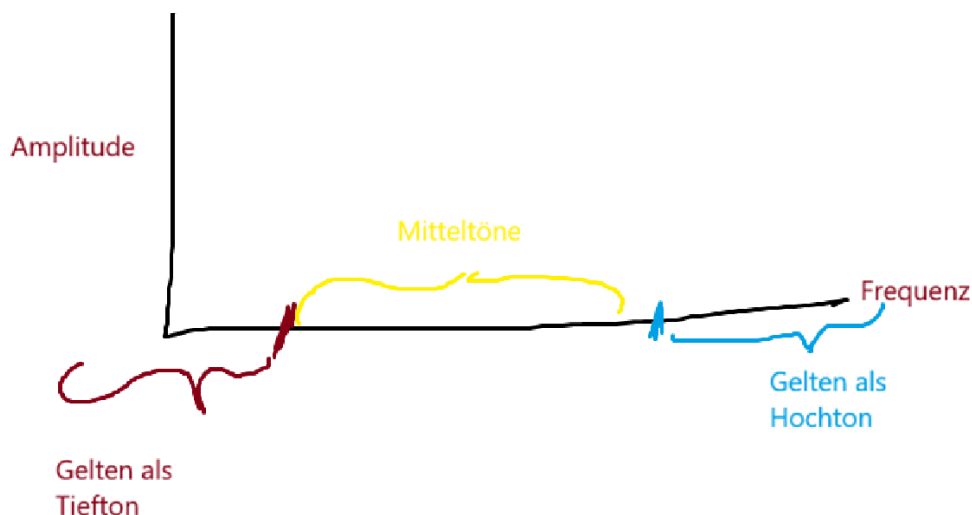
- Abtastrate f_s

$$k = \left\lfloor \frac{f_{\text{Grenze}} \cdot N}{f_s} \right\rfloor$$

→ berechne k_{low} (für Tiefpass) und k_{high} (für Hochpass)

Was wir im prinzip machen ist ein index bestimmen welcher bei einem Amplituden spektrum bestimmt ab wann etwas als Hochtön und als was etwas als Tieftön gilt. Alles dazwischen ist ein Mittelton.

Ungefähr so:



```
In [40]: '''
Die Grenzfrequenzen wurden anhand des Spektrogramms gewählt, da dort die Verteilung
Aufgrund der hohen Energiedichte im unteren Frequenzbereich wurde 600 Hz als Tieffrequenz
von 6 kHz noch relevante Signalanteile vorhanden sind, weshalb diese Frequenz als
Grenzfrequenz gewählt wurde.
Der Mitteltöner ergibt sich als Bandpass zwischen diesen beiden Grenzfrequenzen.
und für jeden Block per FFT in ein Amplitudenspektrum überführt. Die daraus berechneten
Frequenzanteile innerhalb eines Blocks dem Tief-, Mittel- oder Hochtönenbereich zugeordnet.
'''
```

```

import math

tiefpass = 600 #Hz
hochpass = 6000 # Hz

k_low = math.floor((tiefpass * 1024) / 44100)
k_high = math.floor((hochpass * 1024) / 44100)

print(k_low)
print(k_high)

```

13
139

- d. Implementieren drei FFT-Filter: (1) einen idealen Tiefpass; (2) einen idealen Bandpass; (3) einen idealen Hochpass. Dies geschieht durch einfaches Nullsetzen aller Frequenzen im Sperrbereich und Beibehaltung des Spektrums im Durchlassbereich für jeden 1024er-Block. Achten Sie darauf, dass das FFT-Spektrum spiegelsymmetrisch ist, d.h. z.B. bei einem Hochpass müssen Sie das Spektrum für alle Frequenzen von 0 bis zur Grenzwellenzahl k auf Null setzen, aber auch für alle Frequenzen von $N - k$ bis $N - 1$.

```

In [41]: def fft_filter_tiefpass(k_low, blocks):
    # muss man mal schauen wie das hier gemacht wird
    filtered_blocks = [] # Leeres Array wo die gefilterten Blocks sich befinden
    N = len(blocks[0]) # holt die Blockgröße
    dt = 1 / fs # Abtastintervall

    for block in blocks:
        X = np.fft.fft(block) # Fouriertransformation
        X[k_low:N - k_low] = 0 # Tiefpass Filter
        block_filt = np.fft.ifft(X) # Reeler Teil verwenden der FFT
        filtered_blocks.append(np.real(block_filt))

    ret = np.concatenate(filtered_blocks) # setze mehrere Arrays zusammen
    # Zeitachse passend zum Signal erzeugen
    time = np.arange(len(ret)) * dt
    ampspec(signalwert=ret, zeitachse=time)

    return ret

def fft_filter_bandpass(k_low, k_high, blocks):
    # muss man mal schauen wie das hier gemacht wird
    filtered_blocks = [] # Leeres Array wo die gefilterten Blocks sich befinden
    N = len(blocks[0]) # holt die Blockgröße
    dt = 1 / fs # Abtastintervall

    for block in blocks:
        X = np.fft.fft(block) # Fouriertransformation
        X[:k_low] = 0
        X[k_high:N - k_high] = 0
        X[N - k_low:] = 0
        block_filt = np.fft.ifft(X) # Reeler Teil verwenden der FFT
        filtered_blocks.append(np.real(block_filt))

    ret = np.concatenate(filtered_blocks) # setze mehrere Arrays zusammen
    # Zeitachse passend zum Signal erzeugen

```

```

time = np.arange(len(ret)) * dt
ampspec(signalwert=ret, zeitachse=time)

return ret

def fft_filter_hochpass(k_high, blocks):
    filtered_blocks = []
    N = len(blocks[0])
    dt = 1 / fs          # Abtastintervall

    for block in blocks:
        X = np.fft.fft(block)
        X[:k_high] = 0
        X[N - k_high:] = 0
        block_filt = np.fft.ifft(X)
        filtered_blocks.append(np.real(block_filt))

    ret = np.concatenate(filtered_blocks) # setze mehrere Arrays zusammen
    # Zeitachse passend zum Signal erzeugen
    time = np.arange(len(ret)) * dt
    ampspec(signalwert=ret, zeitachse=time)

    return ret

```

e. Überprüfen Sie Ergebnis akustisch durch Abspielen mit `IPython.display.Audio`.

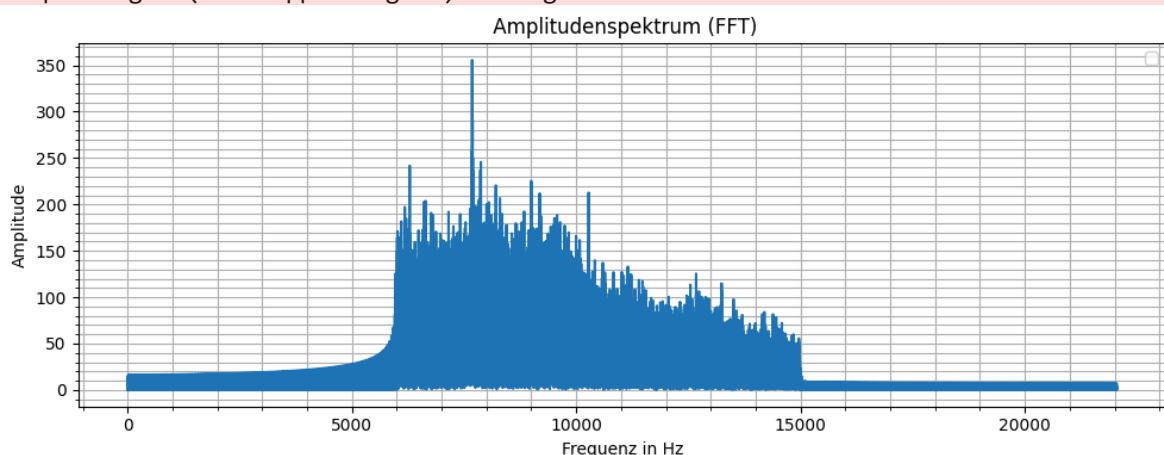
```

In [42]: # Kontrolliere die Amplituden Spektren
print("high")
high = fft_filter_hochpass(k_high, blocks)
print("low")
low = fft_filter_tiefpass(k_low, blocks)
print("mittel")
band = fft_filter_bandpass(k_low, k_high, blocks)

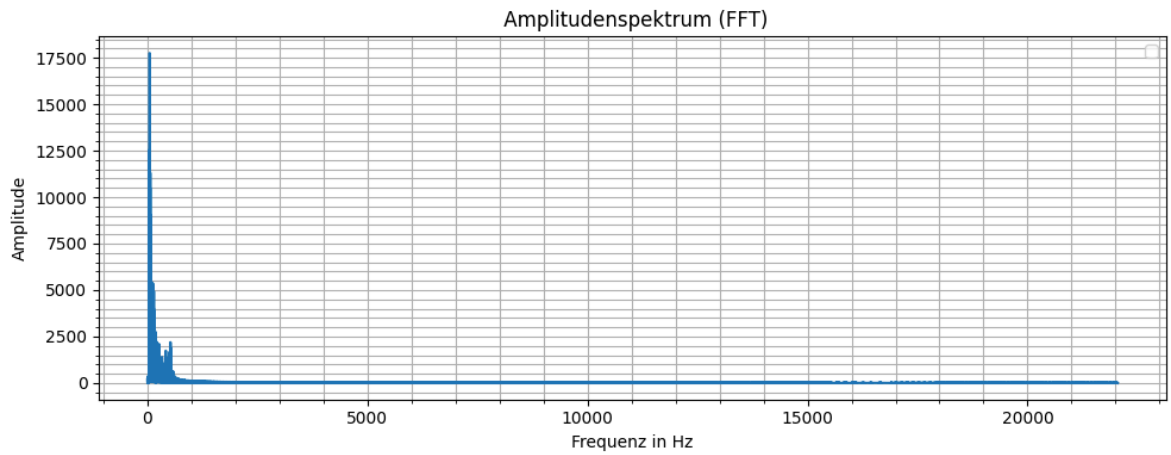
```

high

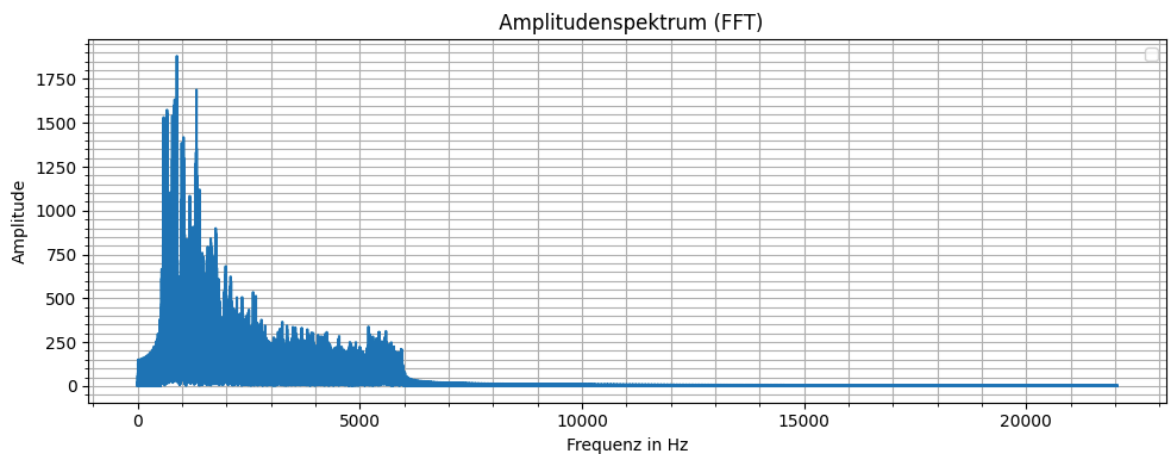
C:\Users\Nikita\AppData\Local\Temp\ipykernel_5920\2100571738.py:36: UserWarning:
No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend(loc='upper right') #legende oben rechts



low



mittel



```
In [43]: # Abspielen vom Hochpass
playAudio(high, fs)
```

```
Out[43]: ▶ 0:00 / 0:09 ————— 🔊 ⋮
```

```
In [44]: playAudio(low, fs)
```

```
Out[44]: ▶ 0:00 / 0:09 ————— 🔊 ⋮
```

```
In [45]: playAudio(band, fs)
```

```
Out[45]: ▶ 0:00 / 0:09 ————— 🔊 ⋮
```

```
In [46]: # Aus reinem interesse, hier noch ein Spektrogramm
std_spectrogram(high, fs)
std_spectrogram(low, fs)
std_spectrogram(band, fs)
```